

PRACTICAL: Linux Process Management and Process Creation in C

1.Which commands will he use to complete the given task with the help of the following operation?

- Kill Processes bY name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

Topic:

Linux provides commands to view running processes and terminate them to reduce system load.

Commands Used:

View all running processes

`ps aux`

Search process by name

`ps aux | grep process_name`

Kill process by name

`pkill process_name`

Kill all processes with same name

`killall process_name`

Kill a single process using PID

`kill PID`

Force kill a process

`kill -9 PID`

Get PID directly using name

`pgrep process_name`

```

m309      8958      2290  0 16:14 ?        00:00:00 /usr/libexec/ModemManager
root      9278        2  0 16:19 ?        00:00:00 [kworker/u16:0-events_
root      9281        2  0 16:20 ?        00:00:00 [kworker/0:1-cgroup_de
root      9282        2  0 16:20 ?        00:00:00 [kworker/3:1-events]
root      9335        2  0 16:24 ?        00:00:00 [kworker/1:0-cgroup_de
m309      9356      2290  0 16:24 ?        00:00:00 gjs /usr/share/gnome-s
root      9408        2  0 16:24 ?        00:00:00 [kworker/2:1-cgroup_de
root      9415        2  0 16:28 ?        00:00:00 [kworker/u16:2-flush-8
root      9421        2  0 16:30 ?        00:00:00 [kworker/2:2]
root      9425        2  0 16:31 ?        00:00:00 [kworker/0:2]
m309      9427      7897  0 16:31 pts/0    00:00:00 ps -ef
m309@m309-BY-OEM ~ % pkill colord
pkill: killing pid 1643 failed: Operation not permitted
m309@m309-BY-OEM ~ % pkill rtkik
m309@m309-BY-OEM ~ % ps -ef|rtkik
zsh: command not found: rtkik
m309@m309-BY-OEM ~ % █

```

```

m309@m309-BY-OEM ~ % kill 8952
m309@m309-BY-OEM ~ % ps -ef|8952
zsh: command not found: 8952
m309@m309-BY-OEM ~ % █

```

2. Write a program for process creation using C

- Orphan Process
- Zombie Process

Topic:

Process creation in Linux is done using the `fork()` system call. Special process states include Orphan and Zombie processes.

Orphan Process:

An orphan process is created when the parent process terminates before the child process. The child process is then adopted by `init/systemd` (PID becomes 1).

Functions Used:

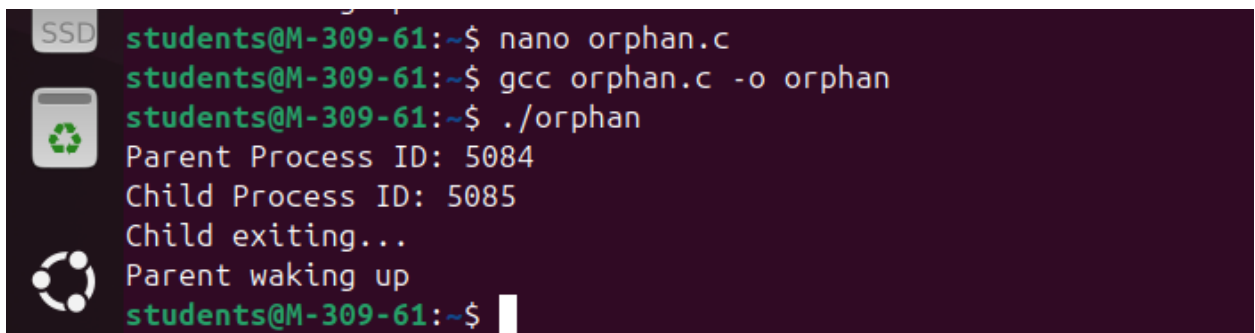
fork()

getpid()

getppid()

sleep()

exit()



```
SSD students@M-309-61:~$ nano orphan.c
students@M-309-61:~$ gcc orphan.c -o orphan
students@M-309-61:~$ ./orphan
Parent Process ID: 5084
Child Process ID: 5085
Child exiting...
Parent waking up
students@M-309-61:~$
```

Zombie Process:

A zombie process is created when a child process finishes execution but the parent does not collect its exit status. The process remains in the process table as defunct.

Functions Used:

fork()

getpid()

sleep()

exit()

```
m309@m309-BY-OEM:~$ nano orphan.c
m309@m309-BY-OEM:~$ nano zombie.c
m309@m309-BY-OEM:~$ gcc zombie.c -o zombie
m309@m309-BY-OEM:~$ ./zombie
Child exiting

Parent sleeping
```

3.Create the process using fork () system call

- Child Process creation
- Parent Process creation

Topic:

The fork() system call creates a new child process. Both parent and child run independently and have different process IDs.

Concepts Demonstrated:

Parent Process

The original process that calls fork().

Child Process

The new process created by fork().

PID

Process ID of a running process, obtained using getpid().

PPID

Parent Process ID, obtained using

getppid().

Core System Call Used

fork()

C Functions Used

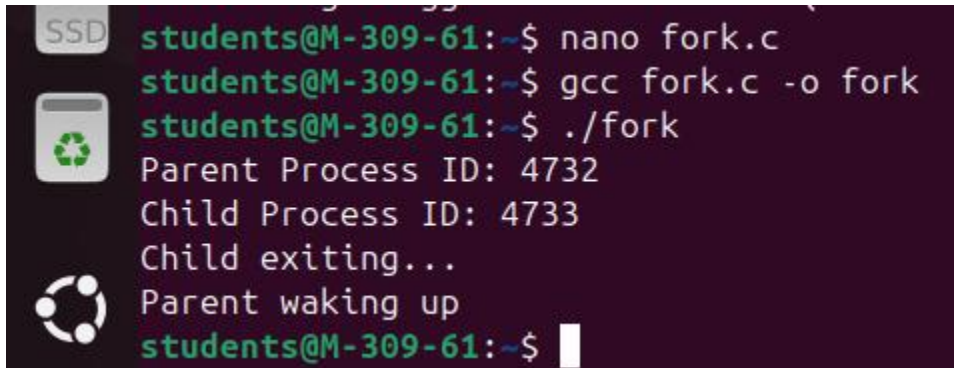
fork() – Creates a child process

getpid() – Returns current process ID

getppid() – Returns parent process ID

sleep() – Delays execution

exit() – Terminates a process



```
SSD students@M-309-61:~$ nano fork.c
students@M-309-61:~$ gcc fork.c -o fork
students@M-309-61:~$ ./fork
Parent Process ID: 4732
Child Process ID: 4733
Child exiting...
Parent waking up
students@M-309-61:~$
```

Compilation and Execution Commands

Create C file

nano program.c

Compile program

gcc program.c -o program

Run program

./program